

CaseWhy — Technical Brief for USCIS Production API Access

Prepared for USCIS Torch Developer Support ahead of the Case Status API production-access demo.

Applicant: CaseWhy LLC (Florida), sole founder and operator: Peter. Development is AI-assisted (Claude Code); every technical claim in this brief was checked directly against the live application code before this document was written, not recalled from memory.

What CaseWhy is: a free web application that lets people track the status of their own USCIS case by receipt number, explains what a status means in plain language, and shows published processing times and Visa Bulletin data. It is not a law firm and gives no legal advice. Live at `app.casewhy.com`.

0. Self-assessment against USCIS's five demo criteria

Checked directly against `developer.uscis.gov`'s own demo-criteria and demo-ID pages, not assumed from an earlier internal list. File upload / encoded-document handling is a FOIA API criterion, not a Case Status API one — it is intentionally kept out of this demo's live flow, and out of this table.

#	USCIS criterion	Status	Where it's demonstrated
1	Data entry usability	Pass	Receipt-number field with real-time format feedback, error copy, sign-in, Spanish toggle, mobile-responsive layout.
2	Application data converts to a JSON payload	Pass	A real "Show technical details" panel on the on-demand check button renders the exact outbound request (method, URL, headers, redacted token) — not just a claim, a live view of the same code path that actually runs.
3	OAuth 2.0 authentication	Pass	Client Credentials grant, token cached and refreshed server-side, credentials never reach the browser.
4			Found during preparation: the on-demand check button was previously swallowing USCIS's real error message behind a generic string. Fixed to surface it directly (<code>src/lib/uscis/client.ts`</code> s

	HTTPS response handling, success and error, message shown in the UI	Pass, fixed same day as this brief	<code>`extractUscisErrorMessage()`</code> , <code>`src/app/dashboard/actions.ts`</code> 's <code>`checkCaseNow()`</code>) before this brief was finalized — see Section 4.
5	Case status tracking	Pass	Add a case, dashboard display, on-demand and scheduled checks, status-history table, change-notification email.

`demo_id: src/lib/uscis/client.ts`'s `demoIdHeader()` sends a `demo_id` header on every Case Status API request whenever `USCIS_DEMO_ID` is set — added specifically for USCIS's screenshot-verification requirement before the scheduler link is issued, and removed after go-live.

1. Architecture and hosting

CaseWhy is a Next.js 15 application (App Router) hosted on Vercel, with PostgreSQL (Neon) for data storage via Drizzle ORM. **[1]** Transactional email runs through Postmark; billing (for the optional paid tier) through Stripe.

Every server route — including every call to USCIS's Case Status API — runs on Vercel's standard Node.js runtime, not the Edge runtime. This was a deliberate choice, not a default: the app's database access needs a real TCP connection pool, which Edge doesn't support, so `middleware.ts` explicitly sets `runtime: "nodejs"`. **[2] The USCIS API client (``src/lib/uscis/client.ts``) runs exclusively on the server. Client ID, client secret, and access tokens never reach the browser** — the browser only ever talks to CaseWhy's own Next.js server, which in turn talks to USCIS.

End-to-end flow for a status check:

1. A signed-in user submits a receipt number (`src/app/dashboard/actions.ts`, `trackCase()`).
2. The receipt number is encrypted and stored (`tracked_cases` table).
3. A scheduled job (`/api/cron/check-status`, currently run by GitHub Actions on a fixed schedule) decrypts the receipt number server-side, calls `getCaseStatus()`, and compares the result to the last known status.
4. On a genuine change, the user is emailed via Postmark and, if enabled, sent a web push notification.

There is no native mobile app; CaseWhy is a installable Progressive Web App only.

2. OAuth 2.0 authentication

CaseWhy uses the **OAuth 2.0 Client Credentials grant**, exactly as documented by USCIS. **[3]** `getAccessToken()` POSTs `grant_type=client_credentials` with the client ID/secret as `application/x-www-form-urlencoded` to the token endpoint, and caches the resulting token in memory keyed by

client ID, refreshing automatically 30 seconds before real expiry rather than on a fixed timer. [4] Sandbox and production use structurally separate credential pairs and separate token/API base URLs, selected by a single `USCIS_ENV` environment variable — there is no code path where sandbox and production credentials can be mixed up. [5]

The Case Status API and the (separately applied-for) FOIA API use two distinct credential pairs, cached under separate keys, since they are two different Torch applications. [6]

All required credentials are read from environment variables and fail loudly (a clear thrown error, not a silent fallback) if missing. [7] No credential is ever committed to source control — `.gitignore` excludes every `.env*` file, and credentials live only as encrypted environment variables in Vercel.

On a 401 or expired token mid-request: the next call simply re-authenticates (the cache check re-evaluates on every call); there is no retry storm, since a failed token fetch throws and the calling job reports that row as an error rather than looping.

3. JSON payload handling

`getCaseStatus()` calls USCIS's Case Status API and parses the response against a typed shape (`RawCaseStatus`) covering both the standard success response and the `IOE`-prefix variant, whose `submittedDate`/`modifiedDate` fields are documented as sometimes absent — handled as optional, not assumed present. [8] Full status history returned by USCIS (`hist_case_status`) is preserved and stored per-entry.

Fields consumed: form type, current status text and description, submitted/modified dates, and the full history array. **The raw receipt number is never sent to any third party beyond USCIS itself** — it is deliberately excluded from the AI-explanation prompt context (see Section 8). [9]

Status text is shown to the user verbatim, exactly as USCIS returns it, visually distinguished from `CaseWhy`'s own plain-language explanation of that text — the app never rewrites or paraphrases USCIS's own words as if USCIS said them.

Unexpected/malformed responses: on a non-OK HTTP status, the client attempts to parse a JSON error body and falls back to the raw response text if that fails — it never throws an unhandled parsing exception on a malformed error payload. [10]

Showing the actual outbound request, not just describing it. `describeCaseStatusRequest()` mirrors exactly what `getCaseStatus()` sends — same URL, same headers — with the token redacted, and is surfaced live in the dashboard's on-demand-check UI behind a "Show technical details" toggle. [10a] It makes no network call of its own, so it can't drift out of sync with what's actually sent.

4. Error handling

A typed `UscisApiError` (carrying HTTP status and USCIS's own error body) is thrown for every non-2xx response and caught explicitly at each call site to produce a readable message. [11] There is no automatic retry logic in the client — a failed call is reported as a failure for that specific check, not silently retried in a loop that could compound load on USCIS's API.

Real error shapes confirmed against the sandbox directly, not assumed from documentation. A malformed receipt number (AB12345) returns a real **422** with `{"message": "The application receipt number is not formatted correctly..."}`; an unknown but well-formed receipt number returns a real **404** with `{"message": "Case Status Online does not recognize the receipt number entered..."}`. Both are a flat `{"message": "..."}` shape in the sandbox today, not an errors array — `extractUscis-ErrorMessage()` handles both that flat shape and an `errors[]`.message array shape defensively, so it degrades gracefully rather than assuming one specific format. **[11a]**

The error message now reaches the UI, not just a log — fixed during preparation of this brief, not assumed already correct. The on-demand check button (`CheckNowButton.tsx`) was found, while verifying this claim, to be replacing USCIS's real error text with a generic "Couldn't reach USCIS right now" string. `checkCaseNow()` now surfaces USCIS's actual message directly to the UI. **[11b]**

Receipt-number format is validated client-side in real time (3 letters + 10 digits, `/^[A-Z]{3}\d{10}$/`) purely as UX feedback — it never blocks submission, since a well-formed number still might not be real. **[12]** Server-side, an invalid or unknown receipt number is simply sent to USCIS and handled as a normal `UscisApiError` — the 422/404 responses above, not a separate guessed code path.

Logging: the scheduled status-check job's own response includes a structured `errors` array (row ID + message) for whatever failed in that run; receipt numbers are never written to logs in plaintext, since the job operates on the decrypted value only in-memory for the duration of the API call.

Known, honest gap: there is no dedicated alerting/dashboard for USCIS API error rates over time beyond that per-run error array and Vercel's own default function logs. This is not overclaimed as monitoring infrastructure that doesn't exist.

5. Rate limiting and volume

Nothing in the API client itself throttles or paces calls — the actual pacing discipline lives in how `CaseWhy` schedules calls. The production status-check job runs on a fixed daily schedule per tracked case (an on-demand "check now" is also available as a paid-tier feature, itself naturally rate-limited by real human clicks). At `CaseWhy`'s current scale this generates a small number of calls per day; the architecture (one scheduled job iterating tracked cases, no per-request client hammering) means volume scales linearly and predictably with the number of tracked cases, not with traffic spikes.

Real, exercised sandbox testing, not just a code review: starting Monday, September 14, 2026, `CaseWhy` ran a dedicated sandbox volume test — see Section 10 for real, current numbers.

A separate, real example of rate limiting elsewhere in the app (illustrating the team's general practice, not USCIS-specific): the anonymous "ask a question" feature enforces a durable, database-backed lifetime cap of 3 questions per IP address — not an in-memory window that resets. **[13]**

6. Case status tracking

A tracked case (`tracked_cases` table) stores an encrypted receipt number, case type, and the last known status text; `case_status_history` stores one row per real USCIS history entry (service-center prefix only — never

the full receipt number a second time). [14] Deduplication is a database-level unique constraint on (tracked-CaseId, statusText, eventDate), safe to re-run on every check including the very first one.

Change detection: the scheduled job decrypts the previous status text, compares it against the freshly fetched one, and — only on a real change, and never on the very first check for a case — sends a Postmark email and (if enabled) a push notification. [15]

Removing a case is self-serve and immediate: `untrackCase()` is a real, ownership-checked SQL DELETE on the `tracked_cases` row, callable from the dashboard at any time. [16] Its `case_status_history` rows (which never contained the full receipt number) are not automatically cascade-deleted with it today — an honest, current limitation, not something this brief claims is otherwise.

Free accounts can track up to 3 cases; the paid tier raises this in tiers up to a hard ceiling, with anything above 25 cases requiring a real conversation rather than a self-serve path. [17]

7. File upload (document vault) — not a Case Status API criterion

Encoded file upload is a FOIA API demo criterion, not a Case Status API one, per USCIS's own published criteria — it is deliberately not part of this demo's live flow, and mentioned here only for completeness. The paid tier does include a real document-upload feature — this is a working implementation, not a placeholder. [18] Uploads accept PDF, JPEG, or PNG up to 10 MB, stored in Vercel Blob under **private access** (never a public URL); the original filename is encrypted before storage, since a filename can itself be identifying. [19] Downloads are proxied through CaseWhy's own server rather than exposing a direct Blob URL. [20] **No uploaded document is ever sent to USCIS or any third party** — the vault exists purely for the user's own reference and for CaseWhy's escalation-letter drafting tool.

8. Privacy, security, and data handling

Receipt numbers are treated as personally identifiable information. They are encrypted with AES-256-GCM (authenticated encryption — tampering is detected, not silently accepted) before being written to the database; the encryption key exists only as a Vercel environment variable, separate from the database itself. [21] No CaseWhy staff member can view a receipt number in plaintext through any administrative tool.

What is sent to the AI service (Claude, via Anthropic, through the Vercel AI Gateway): form type, processing center (derived from the receipt-number prefix), dates, status text/description, full status history, and matched policy background — **the raw receipt number is deliberately never included**, by explicit design in the prompt-construction code, not merely by policy. [22] The one exception: if a user on the paid tier uses the escalation-letter drafting tool, their receipt number is included in the drafted letter text (since the letter has to identify the case to USCIS or a congressional office) and is sent to the same AI service solely to help draft that letter.

Authentication: Neon Auth, supporting both password and magic-link sign-in. [23] **Admin access** is gated by a single allow-listed email (fail-closed: if unset, no one is treated as an admin). [24]

Retention and deletion: removing an individual tracked case is immediate and self-serve. Full account deletion is a request to `privacy@casewhy.com` (one of 13 real, monitored response aliases on the domain), completed within 30 days — this is accurately described in the live Privacy Policy as of this writing, corrected during prepa-

ration of this brief after an earlier version of the policy overstated self-serve deletion as available "from account settings," which was not actually built. [25]

Policy-change acknowledgment: users are required to actively acknowledge a material change to the Privacy Policy or Terms of Use the next time they sign in, not just notified by email. [26]

Known, honest security gaps, not glossed over: no formal automated security-header configuration (CSP, HSTS, etc.) exists in the app's own config today — HTTPS/TLS is provided by Vercel's platform by default, but CaseWhy has not layered custom security headers on top. No formal dependency-vulnerability scanning process exists beyond normal npm tooling. Vercel's own automatic DDoS/bot mitigation sits in front of the domain at the platform level.

9. Business and operations

Monetization: a free tier (3 tracked cases, 3 AI questions/month, full access to all reference data — processing times, Visa Bulletin, Get Help directories) and a paid "Plus" tier (currently \$9.99/month, \$22.99/quarter, or \$66.99/year, confirmed live against the current pricing table as of this writing). [27] **Nothing from the USCIS Case Status API is ever paywalled** — status checking itself is free at every tier; Plus adds more tracked cases, unlimited AI questions, on-demand checks, the document vault, and drafting tools. Billing runs on Stripe, currently in **test mode** — CaseWhy has processed no real production payments to date; flipping to live payments is a distinct, deliberate step gated on completing business banking setup, not yet taken. [28]

Support: 13 real, monitored email aliases on the domain (including security@, privacy@, legal@, abuse@, help@), each polled and AI-drafted for response, with the most sensitive three escalated immediately in addition to the normal queue. [29]

If production API access were ever revoked: CaseWhy has no scraping fallback and does not attempt to work around API unavailability — the app would show an honest degraded-service message; it does not have, and will not build, an unauthorized alternative path to USCIS case data.

10. Sandbox testing summary

Following USCIS's initial rejection of a production-access request for insufficient sandbox traffic (September 11, 2026), CaseWhy ran a dedicated 5-consecutive-business-day sandbox volume test, **Monday September 14 through Friday September 18, 2026**, using the app's real production code path (`getCaseStatus()`), not a mock. [30]

- Target: roughly 50 calls per run (~30 success, ~20 deliberate-error mix), spaced ~1–1.5 seconds apart, 18 runs/day across the sandbox's published 7 AM–8 PM ET operating hours — roughly 900 calls/day, under the sandbox's published 1,000-request/day, 5-transactions-per-second ceiling.

- A real correction made early in the test, not assumed: the sandbox does not mock arbitrary well-formed receipt numbers — it serves a small, curated pool of specific (prefix, digits) combinations across 7 form types, discovered by a live probe before committing further volume to a wrong assumption.

- As of Wednesday, September 16, 2026 (the most recent completed run at the time of writing): every run across the test window has completed with **zero quota-exceeded responses and zero unexpected errors** — the daily target has been met on every business day so far, cleanly.

[Final tallies for the full 5-day window will be appended here once Friday, September 18's runs complete.]

-
- [1] package.json; src/lib/db/client.ts.
 - [2] src/middleware.ts; next.config.ts (experimental.nodeMiddleware).
 - [3] src/lib/uscis/client.ts, header comment and getAccessToken().
 - [4] src/lib/uscis/client.ts, in-memory token cache keyed by client ID, 30s expiry buffer.
 - [5] src/lib/config.ts, getUscisConfig(), USCIS_ENV.
 - [6] src/lib/config.ts, getUscisFoiaConfig().
 - [7] src/lib/config.ts, required().
 - [8] src/lib/uscis/client.ts, RawCaseStatus type.
 - [9] src/lib/ai/case-context.ts, buildCaseContext() and its own doc comment.
 - [10] src/lib/uscis/client.ts, uscisRequest().
 - [10a] src/lib/uscis/client.ts, describeCaseStatusRequest(); src/app/dashboard/CheckNowButton.tsx.
 - [11] src/lib/uscis/client.ts, UscisApiError; src/app/api/cron/check-status/route.ts.
 - [11a] src/lib/uscis/client.ts, extractUscisErrorMessage(); confirmed live against the real sandbox for both a malformed (422) and unknown (404) receipt number.
 - [11b] src/app/dashboard/actions.ts, checkCaseNow(); src/app/dashboard/CheckNowButton.tsx.
 - [12] src/lib/uscis/receipt-number.ts; src/app/dashboard/ReceiptNumberInput.tsx.
 - [13] src/lib/get-help/anonymous-usage.ts, LIFETIME_CAP_PER_IP.
 - [14] src/lib/db/schema.ts, trackedCases and caseStatusHistory tables.
 - [15] src/lib/uscis/check-status.ts, checkTrackedCaseNow().
 - [16] src/app/dashboard/actions.ts, untrackCase().
 - [17] src/lib/billing/tier.ts, TIER_LIMITS, PLUS_HARD_CEILING_MAX_CASES.
 - [18] src/app/api/documents/route.ts.
 - [19] src/lib/db/schema.ts, caseDocuments table.
 - [20] src/app/api/documents/[id]/route.ts.
 - [21] src/lib/db/crypto.ts.
 - [22] src/lib/ai/case-context.ts.
 - [23] src/lib/auth/server.ts; src/app/auth/sign-in/page.tsx.
 - [24] src/lib/auth/admin.ts, isAdminEmail().
 - [25] Live Privacy Policy, Section 7, www.casewhy.com/privacy.html.
 - [26] src/lib/policy/versions.ts; the acknowledgment-gate middleware.
 - [27] src/lib/billing/pricing.ts, getAllEffectivePrices(), queried live against production.
 - [28] src/lib/stripe/client.ts; Stripe account mode.
 - [29] scripts/seed-email-alias-configs.ts; email_alias_configs table, queried live (13 rows).
 - [30] src/app/api/cron/uscis-volume-test/route.ts; src/lib/uscis/sandbox-volume-test.ts.